

Debugging Guide

This lab uses Linux-native exploit development tools instead of Python web debugging.

The normal compose file already starts the vulnerable services with `SYS_PTRACE` and `seccomp:unconfined`, and the attacker image includes:

- `gdb`
- `pwndbg`
- `checksec`
- `ROPgadget`
- `ropper`
- `pwntools`

Start Debug-Friendly Mode

The debug overlay is intentionally small. It keeps the attacker terminal usable for GDB and preserves ptrace permissions:

```
docker compose -f docker-compose.yml -f docker-compose.debug.yml up --build -d
```

Enter the Attacker Container

```
docker exec -it attacker bash
```

The target binaries are automatically copied into:

```
/targets
```

Check them:

```
ls -la /targets
checksec --file /targets/level1
checksec --file /targets/level2
checksec --file /targets/level3
checksec --file /targets/safe_server
```

Crash-First Debugging

The vulnerable servers install a crash handler that prints register state to Docker logs.

Trigger a crash:

```
python3 scripts/01_fuzzer.py
python3 scripts/02_find_offset.py
```

Read the crash details from the host:

```
docker logs victim-l1 2>&1 | tail -80
```

Look for:

- `RIP`

- RSP
- RBP
- stack bytes near RSP

GDB Static Analysis

Run GDB against the copied binary:

```
gdb /targets/level1
gdb /targets/level2
gdb /targets/level3
```

Useful commands:

```
info functions
disassemble handle_trun
disassemble do_vulnerable_copy
checksec
vmmmap
```

Gadget Discovery

```
ROPgadget --binary /targets/level2
ropper --file /targets/level2 --search "pop rdi"
ROPgadget --binary /targets/level3
```

If the binary does not contain the gadget you want, search libc:

```
ROPgadget --binary /targets/libc.so.6 --only "pop|ret"
```

Safe Server Verification

```
python3 scripts/06_verify_safe.py
```

Expected result: oversized `TRUN` payloads are rejected, format string payloads are printed as text, and the server remains alive.